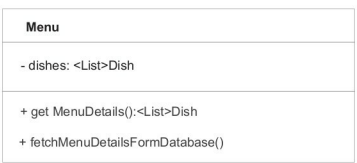

Figure 1: Class diagram for 'Menu'

The problem here is that on every subsequent visit to the *Menu* page, even though the menu details have been fetched once by the same client, the program fetches them again and stores the details in a new object. Is there any way that we can reduce the number of objects created for this class to one, so that this object can be shared across all the classes?

One solution is to make the object global. But this is not a recommended practice in object-oriented programming as this would require tracking of all the global objects and their current status in the program. Also, it does not support the principle of encapsulation.

The GoF introduced the Singleton pattern, wherein only one instance/object of a class is created. This operation is performed in a static member function of the class. The constructor of the class is made private so that this class cannot be instantiated from anywhere else except from within the class itself. The class diagram is shown in Figure 2. As you can see, the instance named 'singleton' is made private and static (underlined), and *getInstance()* is again a static member function that creates and returns the instance of the class.

The *getInstance* member function will first check whether the instance is null or not. If it is null, it creates a new object. Otherwise it returns the existing object. The same process can be used to limit the number of objects to any given number by using a counter to keep track of the number of objects.

There are two ways of instantiating the class – eager instantiation and lazy instantiation.

In eager instantiation, the object is created when the class is loaded. Therefore, irrespective of whether the object is used or not, it is created. The Java code for eager instantiation is given below:

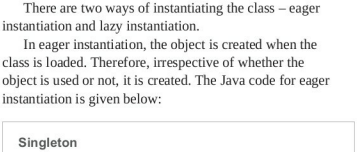

Figure 2: Class diagram for the Singleton design pattern (Source: *https://en.wikipedia.org/wiki/Singleton_pattern*)

```
public class EagerInitializedSingleton {

    private static final EagerInitializedSingleton instance =
new EagerInitializedSingleton();

    //private constructor to avoid client applications to use
constructor
    private EagerInitializedSingleton(){}

    public static EagerInitializedSingleton getInstance(){
        return instance;
    }
}
```

On the other hand, lazy instantiation creates an object only when the static member function is called, as shown below:

```
public class LazyInitializedSingleton {
    private static LazyInitializedSingleton instance;
    private LazyInitializedSingleton(){}

    public static LazyInitializedSingleton getInstance(){
        if(instance == null){
            instance = new LazyInitializedSingleton();
        }
        return instance;
    }
}
```

Anyone who plans to implement this pattern in their program needs to take care of two major points:

- The constructor needs to be made private so that in no circumstances can the class be instantiated from any other class.
- While creating child classes of the singleton class, the instance must be initialised with the instance of the corresponding child class. END

References

'Design Patterns: Elements of Reusable Objected Oriented Software' by Erich Gamma, Richard Helm, Ralph Johnson and John M. Vlissides

[1] *https://en.wikipedia.org/wiki/Singleton_pattern*

[2] *https://www.journaldev.com/1377/java-singleton-design-pattern-best-practices-examples;*

[3] *https://stackoverflow.com/questions/5427818/singleton-and-inheritance-in-java;*

By: Gayatri Venugopal Wairagade

The author teaches programming and Web technologies. Her areas of interest are accessibility, Hindi text processing and education technology. She can be reached at *gayatrivenugopal3@gmail.com.*